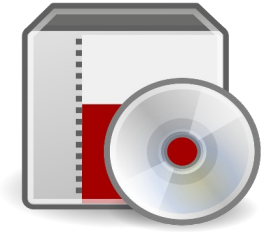
Software Technology 03

Traditional Software Development Models
and Methodologies

Types of software are different

The goal is to make a Software, but is it a

- Product?



- Service?

- Infrastructure?

Developers are Different

Tasks are Different

Consider managing and estimating the following

- Trendy, design oriented web development company (making web shops)
- Outsourcing company delivering information systems
- Game developer company delivering on multiple platforms

Some basics

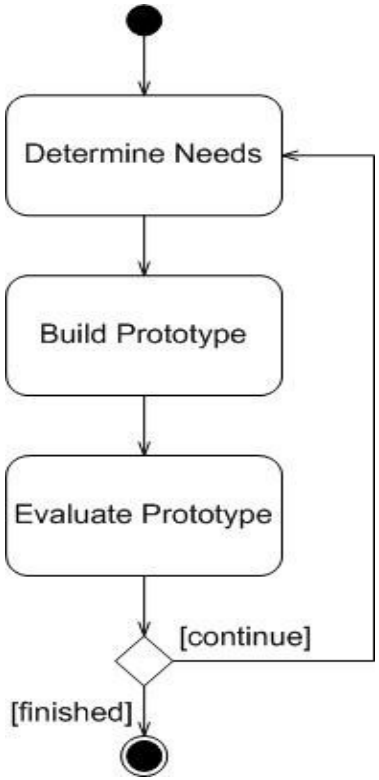
Prototyping

Prototypes are

- Horizontal (shallow, for User, GUI only)
- Vertical (drill down in a function / feature)

Prototyping can be

- Throwaway
- Evolutionary (prototype becomes product)
- Merging (or Incremental)



Prototyping

Good

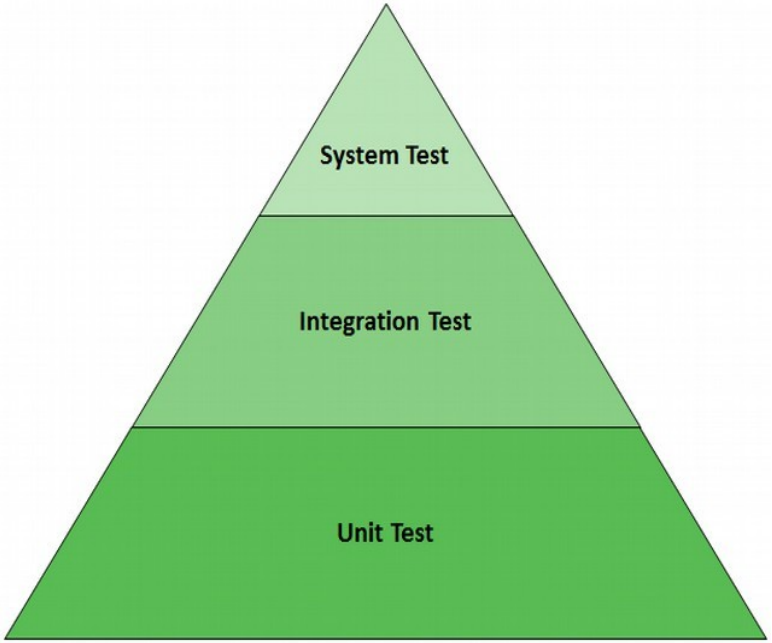
- Fast
- Cheap
- Quick User Feedback
- User/Customer Involvement

Bad

- No planning → problems later or too much iterations
- Too much iterations → Too much prototypes → becomes Expensive and too Long
- Developers dislike throwing away code
- Developers do not understand the real goals (they just have shallow GUI prototypes)

Testing (QA)

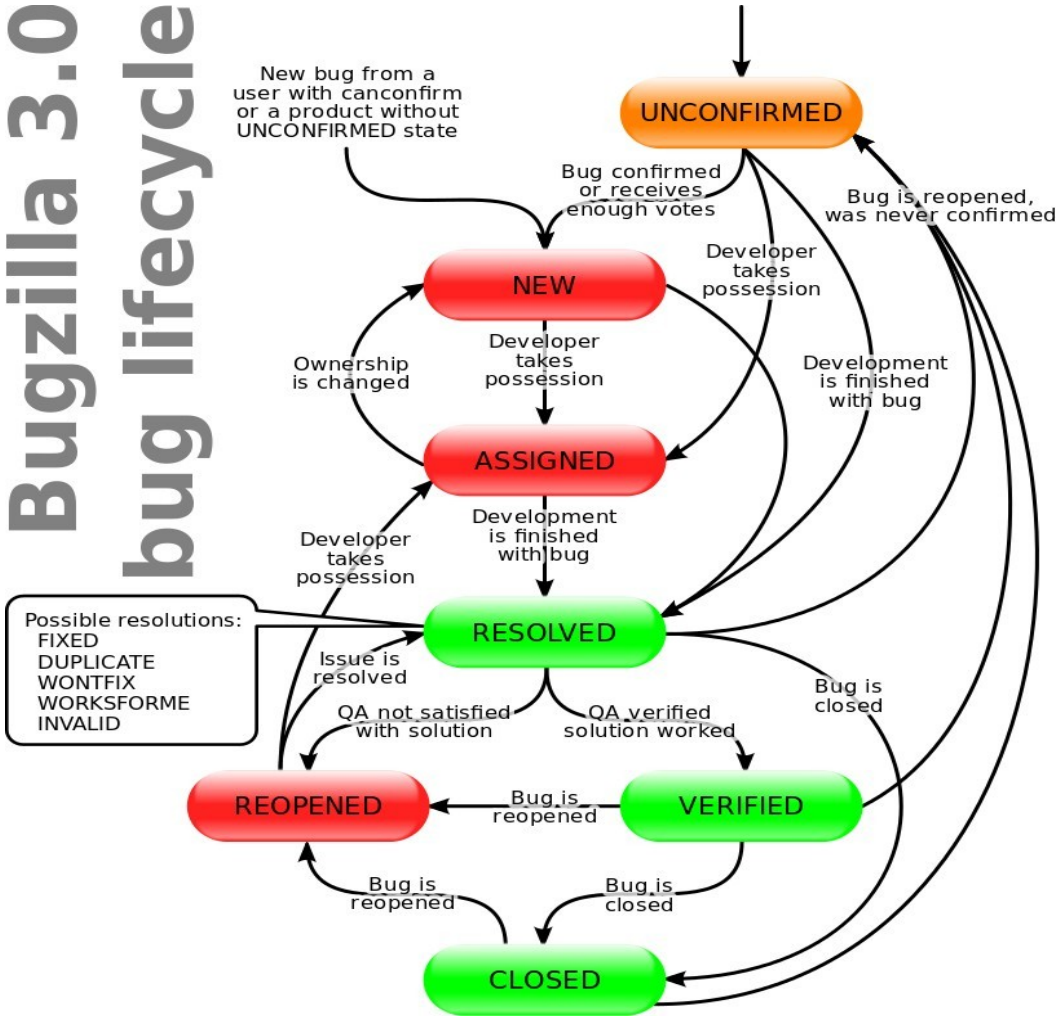
- Unit / Integration (+interface) / System testing
- White-box / Black-box / (Gray-box) testing
- Smoke testing
- Acceptance testing
- Regression testing
- Alpha / Beta testing
- Usability / Accessibility testing
- Performance testing
- Destructive (Stress and Crash) testing
- Security testing
- A/B testing



Issue Tracker Bug Life-cycle

Tools:

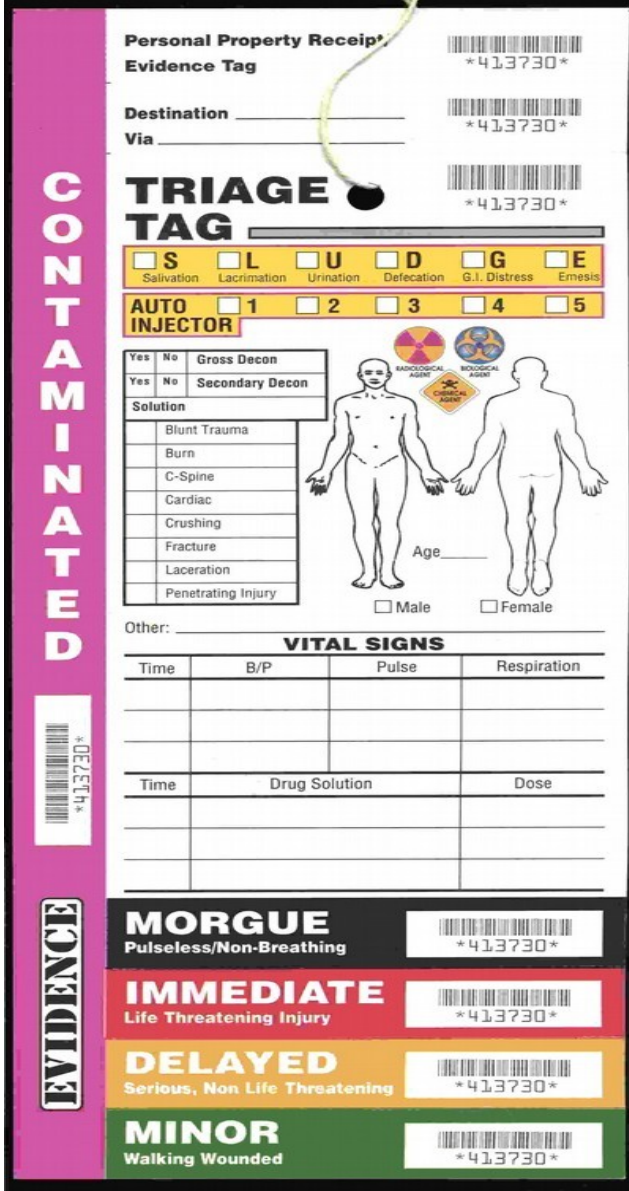
- Trac
- Mantis Issue
- ...



Bug Triage

- Collect information (enough?)
- Identify duplicates / similar problems
- How to reproduce?
- Set Priority + Importance
- Assign

- Fast paced
- Face-to-face (real-time)
- (Support levels)



Time Management Matrix

- Stephen Covey

	Urgent	Not Urgent
Important	Crying baby Kitchen fire Some calls 1	Exercise Vocation Planning 2
Not Important	3 Interruptions Distractions Other calls	4 Trivia Busy work Time wasters

	URGENT	NOT URGENT
IMPORTANT	DO IT NOW	PLAN IT
NOT IMPORTANT	DELEGATE	DROP IT

Software development models: Some (almost) anti-patterns

Cowboy Coding or Code-and-fix

The Anti-pattern

- Coding immediately
- Bugs come, we fix them
- Sometimes we test
- Sometimes we release

Chaos model

- Almost anti-pattern
- Lightweight model
- Definition, implementation and integration on all levels respectively
- Issue is just an incomplete, not well-defined task
- All **issues** are executed according to **categorization**
  - **Value** (to the user)
  - **Urgency**
  - **Robustness** (completeness)

Software development models: Some old-school ones

History of Product Handling and Optimization

We force software to be a Product →

- Division of Labor
- Mass Production
- Interchangeable Parts
- Specialization →

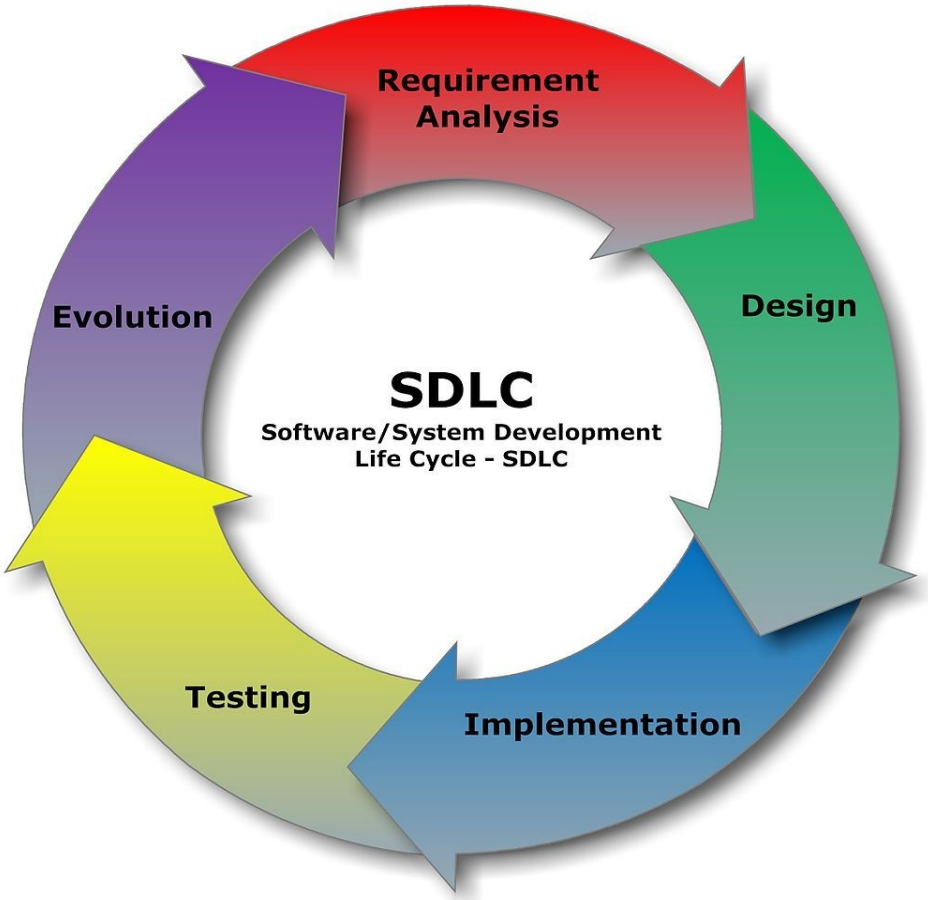
  The Assembly Line / Production Line

- Predictability

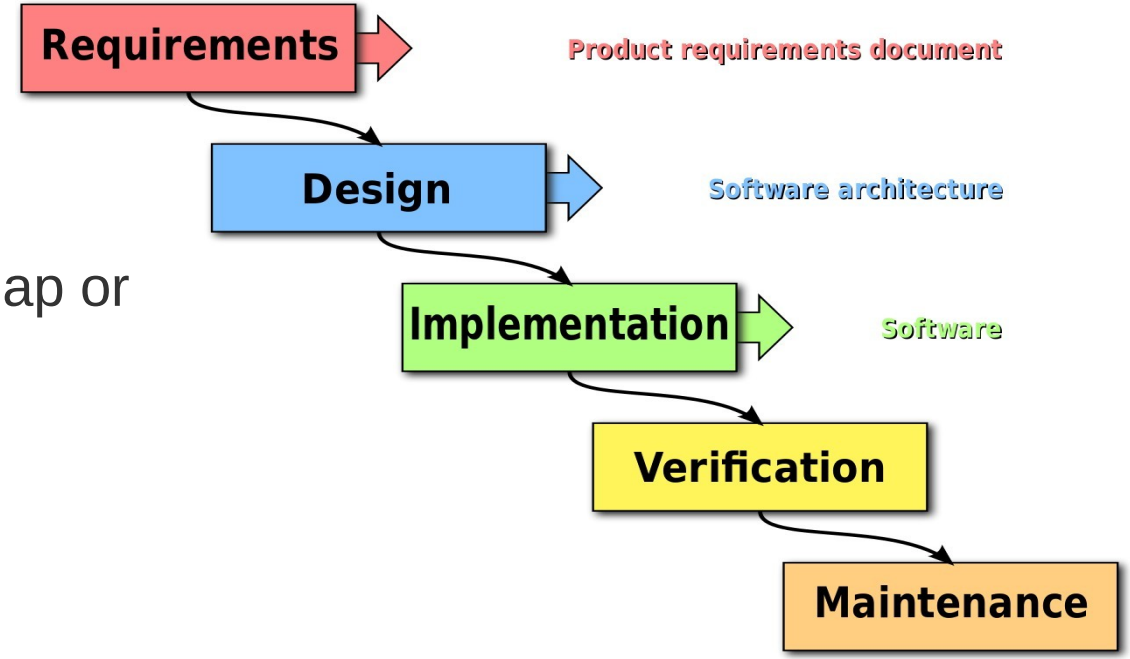
Assembly Line Features

- Consists of Phases
  - Phase uses Results of Previous Phases
- High Quality (involves separate QA phases)
  - QA checks, but quality comes from …?
- Based on Customer Requirements
- Predictably meets
  - Time Schedules
  - Cost Estimates

SDLC


Requirement Analysis
Design
Implementation
Testing
Evolution
SDLC
Software/System Development Life Cycle - SDLC

Waterfall model



- Sequential phases, little overlap or feedback
- Schedules and target dates
- Implement all at once
- Tight control
- Document everything
- Big Design Up Front approach

Waterfall model

Good

- Detailed analysis and planning reduces cost on later stages, may spot problems early
- Only 30% - 40% is implementation
- Structured approach leads to
  - Refined organization structure
  - Detailed documentation
  - Easy recovery (on team member change)
- Easy to monitor (everybody understands where we are)
- Works well if requirements, scope, technology is well understood

Bad

- If something is not understood well, like
  - Requirements
  - Scope
  - Technology
  - Goals, targets
- Too rigid, cannot handle problems with
  - Design
  - Major testing issues

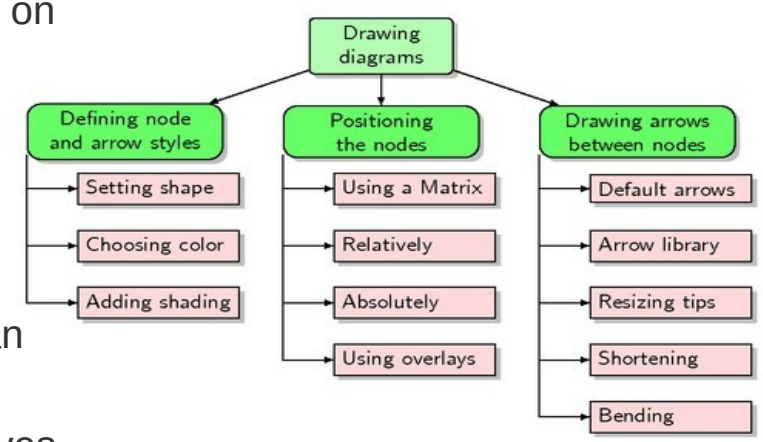
  Multiple things has to be redone then

→ Modified Waterfall models

WBS

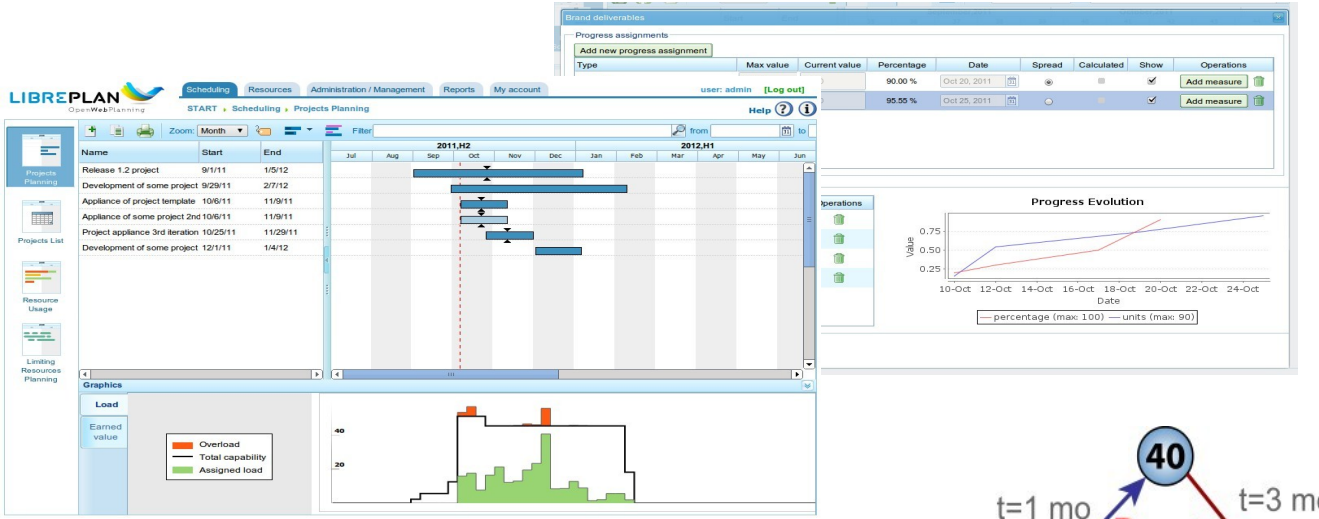
Measurement and control is important, but can only be done on manageable parts→

- **Work Breakdown Structure**
  - Hierarchical (**Tree**) structure
  - Contains the whole **Scope** of the Project
  - Only tells scope, not schedule or detailed execution plan (resource allocations)
  - Breakdown is **Complete** (parent element equals all leaves added)
  - Declares acceptance criteria / **outcomes**, not actions
  - Breaks down until work unit
    - Can be reliably **estimated**
    - Is **measurable**
    - Fits in a reporting period



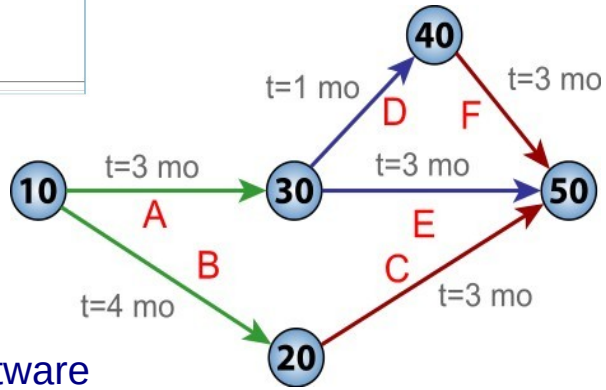
Tools for WBS and beyond

- Trac
- GanttProject
- Calligra Plan
- LibrePlan
- ...



See also:

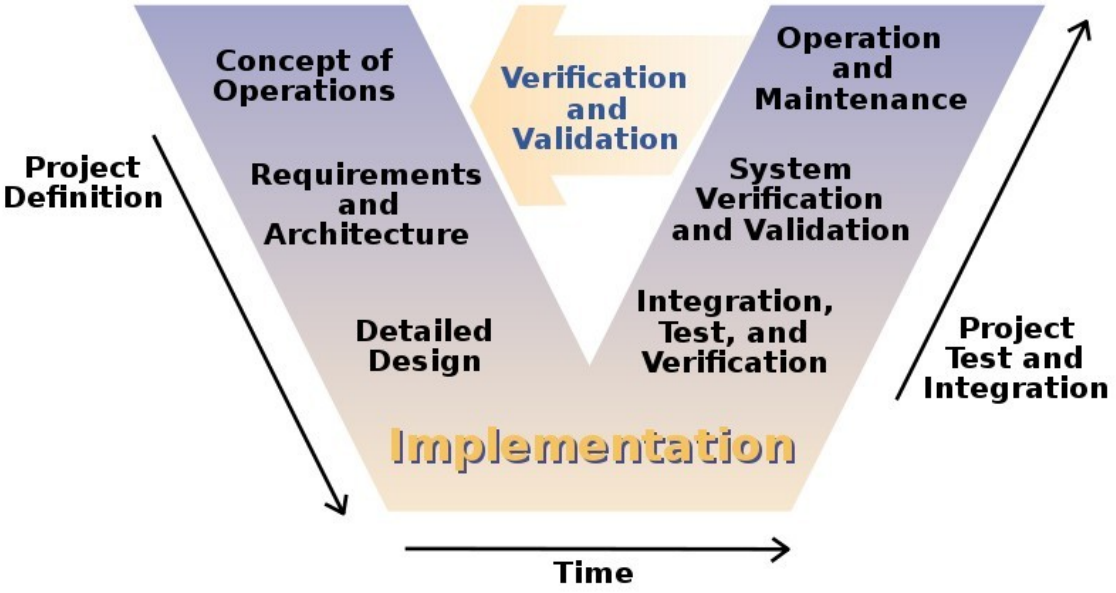
- Critical Chain Project Management
- Critical Path Method
- PERT (Project Evaluation and Review Technique)
- Gantt Chart



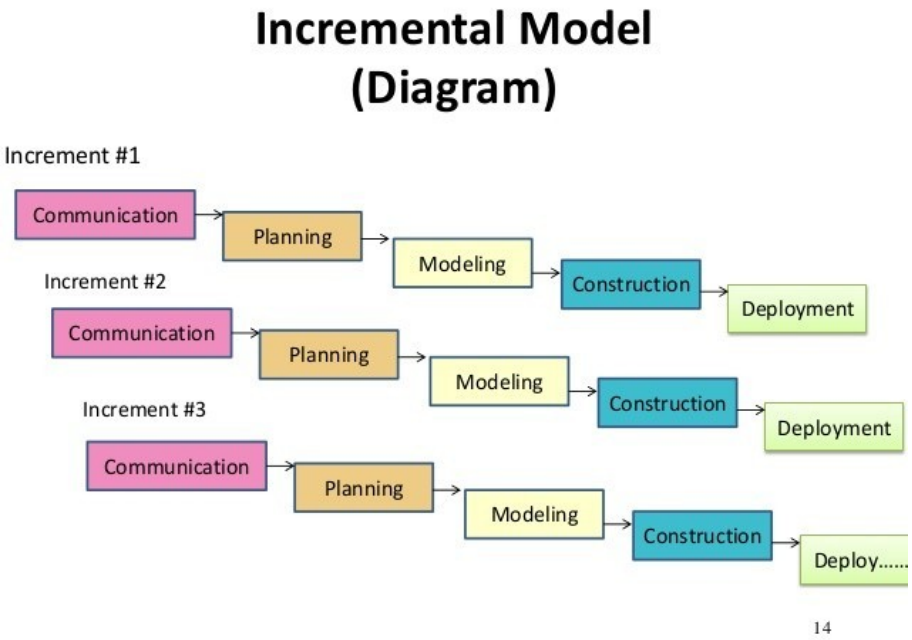
https://en.wikipedia.org/wiki/Comparison_of_project_management_software

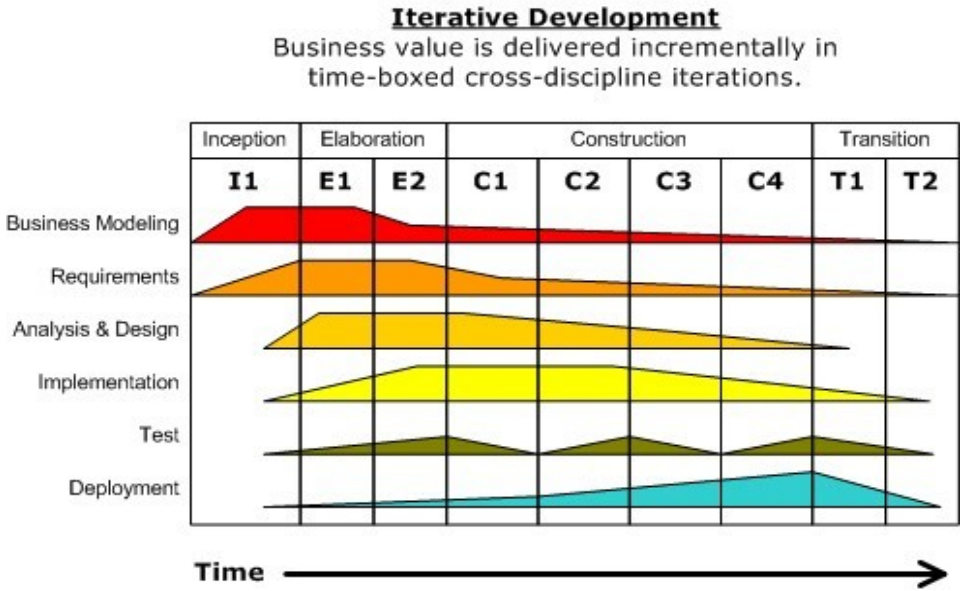
V-Model

- Only A Modified Waterfall
- Has different Levels of Abstraction
- Pairs development and testing phases (but do not unifies them)



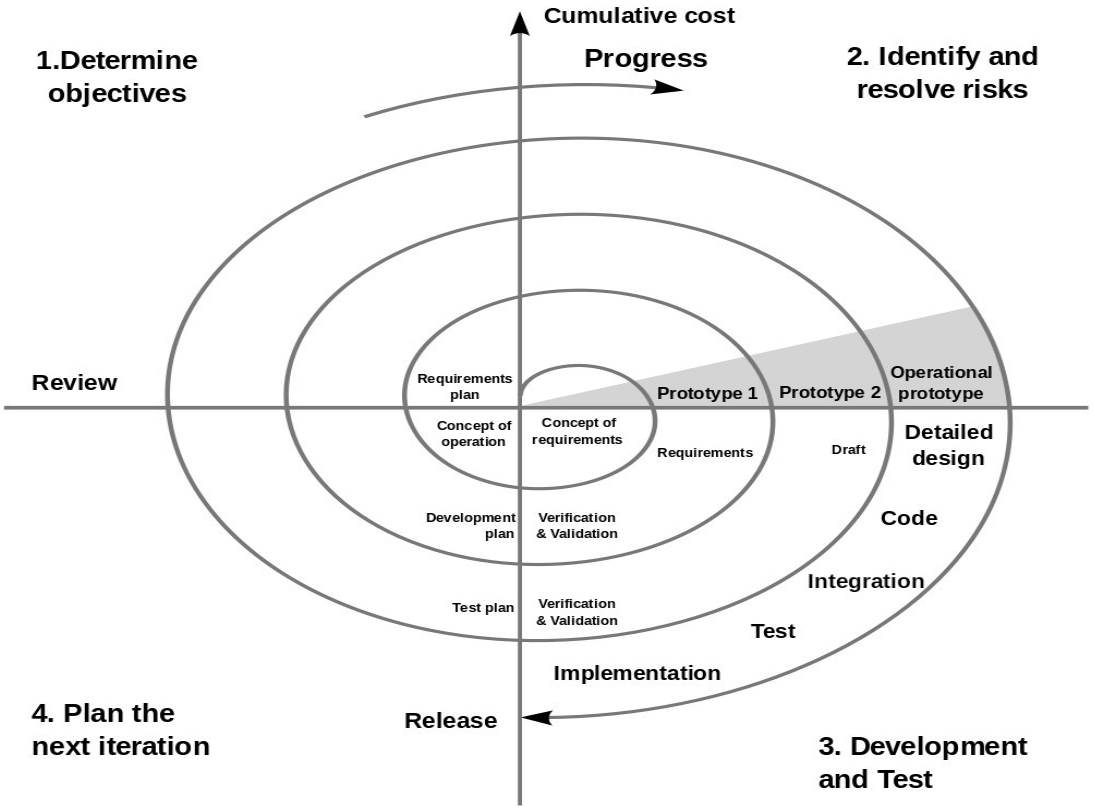
Going Incremental





Spiral model

- = Waterfall + Prototyping + Incremental
- Stakeholder perspective!
- Adaptive in
  - Planning depth (more risk, more detail)
  - Effort (more risk, more effort)



SDLC Criticism

- **Good** in
  - Large projects, complex solutions
  - Detailed, well documented
  - Ease of maintenance
  - Can follow (and certify) standards
- **Bad** in
  - Time and cost is increased
  - Not flexible, cannot handle problems without full-design-first approach
- There are other life cycle approaches
  - Project Life Cycle
  - Application Life Cycle
- Example: SSADM (Structured Systems Analysis and Design Method)
  - Takeaway: Works well on similar problems / projects (in this case information systems)

SDLC vs Assembly Line Analogy

- Assembly Line Features:
  - Every Product is exactly the same we make
  - All work phases are the same, we know
    - How long do they take
    - How much do they cost
  - Workers know everything after training, no further self-development is needed

→ Process risk is very low

NOT TRUE for SW Development!!!